Commute Optimizer — Implementation Report

1. Objective

This project optimizes a complete commute configuration $x = \langle P, t_{start} \rangle$: a directed, loop-free route and a discrete departure time. Travel time is evaluated by the supplied FIFO simulator, which advances the clock edge by edge and applies the Gaussian congestion multiplier at each edge's actual entry time.

2. Operator design and route validity

Neighborhood generation

`get_neighbors` produces two classes of moves:

1. **Departure-time mutation:** shift departure by one legal time step, either $+\Delta t$ or $-\Delta t$.
2. **Local route detours:** choose a contiguous section of the current route and replace it with a short alternative directed path between the same boundary nodes. The detour search uses depth-first enumeration with a bounded length and excludes nodes already used by the unaffected prefix or suffix.

Each candidate is passed through `CommuteProblem.is_valid_state`. This checks the origin and destination, directed-edge continuity, absence of repeated nodes, and the departure-time bounds. A set removes duplicate configurations.

Crossover

`crossover` finds shared intermediate nodes and attempts to splice the prefix of parent 1 with the suffix of parent 2 at a common junction. Every proposed child is validated before it is returned. If the parents have no shared intermediate node, or every splice is invalid, the operator keeps the lower-cost parent's route and uses the nearest legal grid point to the parents' mean departure time. This fallback avoids disconnected or cyclic children.

3. Algorithm implementations

- **Hill Climbing:** evaluates the neighborhood and takes a lowest-cost improving state. Equal-cost sideways moves are allowed up to `max_sideways`; a per-restart visited set prevents cycling. Up to `max_restarts` randomized valid configurations explore other basins.
- **Simulated Annealing:** samples one neighbor at each iteration. Improvements are accepted; a worse move is accepted with probability $\exp(-\Delta E/T)$. Temperature follows $T_{k+1} = \alpha T_k$.
- **Local Beam Search:** holds up to K configurations, pools successors from all beam members, and selects the best K collectively. The selection first favors distinct route structures, then fills remaining slots with the next-best distinct states. This reduces immediate population collapse.
- **Genetic Algorithm:** initializes a population of valid states. Tournament selection maximizes fitness $1/(1 + f(x))$, crossover combines parent routes at shared junctions, mutation selects a random valid neighbor, and elitism copies the best individuals unchanged to the next generation.

All solvers keep the best objective value seen in `self.history`; `iterations` records the number of solver steps executed.

4. Execution and reproduction

The five requested implementation modules retain their original function and class interfaces. Their `TODO` markers remain in place to identify the completed implementation points. The implementation files contain no explanatory code comments.

Run all four algorithms and save the convergence and route figures:

```
python main.py --algo all --seed 42 --plot-metrics --save-figs
```

Reproduce the five-seed runtime summary used for the comparison table:

```
python benchmark.py
```

The benchmark writes `benchmark_results.csv`. The included `convergence_comparison.png` was generated by the all-algorithm run with seed 42. The included `core/`, `visualization/`, and default toy map/scenario are runnable support copies for this standalone bundle. When applying the work to the course repository, replace only the five requested files inside `algorithms/` and the updated `config.json`; preserve the repository's original `core/`, `visualization/`, and data files.